РЕШЕН ИЗПИТ 1 — С ПЪЛНИ ОБЯСНЕНИЯ

Базови Програмни Езици — C (plain C)

Изпит 1: Аптека — Лекарства | 75 точки

Теми: struct + fscanf от текстов файл | намаляване с 20% | fwrite с условие | изтриване от масив

Специфика: long long за 13-цифрен код. Изтриването е shift-left шаблон.

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ

КОД / ИЗРАЗ	КАКВО ОЗНАЧАВА
`наименование (до 30 символа)`	char name[31]; — 30+1 за '\0'
`дата ММ.ГГГГ (фиксиран)`	char expiry[8]; — 07.2025 = 7 символа + '\0'
`13-цифрен код`	long long code; — int не побира 13 цифри!
`единична цена`	float price; — %.2f при printf
`налично количество`	int quantity;
`Прочетете данните от файла`	fscanf() в while цикъл — не fread
`намалява с 20%`	price *= 0.80f;
`записва в offer.bin`	fwrite() с "wb"
`изтрива от масива`	shift-left: arr[i] = arr[i+1] в цикъл

СЕКЦИЯ 1 — ПЪЛЕН КОД (gcc -Wall -std=c99)

> **ВАЖНО:** long long е задължително за 13-цифрен код. %lld е форматът за long long в scanf/printf.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char      name[31];    /* 30 + 1 */
    char      expiry[8];   /* ММ.ГГГГ = 7 + 1 */
    long long code;        /* 13 цифри — задължително long long! */
    float     price;
    int       quantity;
} Medicine;

/* — Задача 1: четене от текстов файл — */
Medicine* loadFile(char *fn, int *count) {
    FILE *f = fopen(fn, "r");
    if (!f) { printf("Error\n"); *count = 0; return NULL; }

    Medicine *arr = NULL;
    *count = 0;
    Medicine tmp;
    while (fscanf(f, "%30[^;];%7[^;];%lld;%f;%d\n",
           tmp.name, tmp.expiry, &tmp.code,
           &tmp.price, &tmp.quantity) == 5) {
        (*count)++;
        arr = realloc(arr, (*count) * sizeof(Medicine));
        if (!arr) { printf("Error\n"); fclose(f); return NULL; }
        arr[(*count) - 1] = tmp;
    }
    fclose(f);
    return arr;
}

/* — Задача 2: намаляване с 20% — */
void discount(Medicine *arr, int count, char *date) {
    int found = 0;
    for (int i = 0; i < count; i++) {
        if (strcmp(arr[i].expiry, date) < 0) {
            float old = arr[i].price;
            arr[i].price *= 0.80f;
            printf("%s - %s - %.2f - %.2f\n",
                   arr[i].name, arr[i].expiry, old, arr[i].price);
            found = 1;
        }
    }
    if (!found) printf("Няма лекарства с изтичащ срок.\n");
}

/* — Задача 3: запис в бинарен файл с условие — */
void saveOffer(Medicine *arr, int count, int minQty) {
    FILE *f = fopen("offer.bin", "wb");
    if (!f) { printf("Error\n"); return; }
    for (int i = 0; i < count; i++)
        if (arr[i].quantity > minQty)
```

```c
            fwrite(&arr[i], sizeof(Medicine), 1, f);
    fclose(f);
}

/* — Задача 4: изтриване по код — */
void deleteByCode(Medicine *arr, int *count, long long code) {
    int idx = -1;
    for (int i = 0; i < *count; i++)
        if (arr[i].code == code) { idx = i; break; }
    if (idx == -1) { printf("Не е намерено.\n"); return; }
    for (int i = idx; i < *count - 1; i++)
        arr[i] = arr[i + 1];
    (*count)--;
}

int main() {
    int count = 0;
    Medicine *arr = loadFile("medicines.txt", &count);
    if (!arr) return 1;
    discount(arr, count, "01.2025");
    saveOffer(arr, count, 10);
    deleteByCode(arr, &count, 1234567890123LL);
    free(arr);
    return 0;
}
```

СЕКЦИЯ 2 — ОБЯСНЕНИЯ РЕД ПО РЕД

Структурата — защо long long?

int побира до ~2 милиарда (10 цифри). long long побира до ~9.2×10^{18} (19 цифри). За 13-цифрен код int ще препълни.

```
[NE]  int code;        // max 2,147,483,647 — само 10 цифри. НЕ е достатъчно!
[OK]  long long code; // побира 13-цифрен код без проблем
```

loadFile() — ред по ред

КОД	ОБЯСНЕНИЕ
`fopen(fn, "r")`	"r" = четене. Не "rb" — файлът е текстов.
`Medicine tmp;`	Временна структура. Четем в нея и после копираме в масива.
`fscanf(f, "%30[^;];..."`	%[^;] чете до символа ';'. %lld е форматът за long long. == 5 проверява дали всички полета са прочетени.
`arr = realloc(arr, ...)`	ЗАДЪЛЖИТЕЛНО arr = realloc. Без arr = губим новия адрес.
`arr[(*count) - 1] = tmp;`	Копираме целия struct с =. Работи защото полетата са фиксирани масиви.

discount() — strcmp за сравнение на дати

Датите са в формат ММ.ГГГГ. strcmp() сравнява лексикографски — '07.2024' < '01.2025' → true → изтекъл срок.

КОД	ОБЯСНЕНИЕ
`strcmp(expiry, date) < 0`	< 0 означава: expiry идва ПРЕДИ date. '07.2024' е преди '01.2025' лексикографски.
`float old = arr[i].price;`	Запазваме старата цена ПРЕДИ да я променим — за принтирането.
`arr[i].price *= 0.80f;`	*= 0.80 = умножение по 0.80 = намаляване с 20%. 'f' суфиксът = float константа.

deleteByCode() — shift-left шаблон

Изтриването от масив в C = преместваме всеки елемент след изтрития с 1 позиция назад (shift left), след което намаляваме брояча.

```
/* Преди: [A][B][C][D]  idx=1 (B се изтрива)      */
/* i=1:   arr[1]=arr[2]  →  [A][C][C][D]          */
/* i=2:   arr[2]=arr[3]  →  [A][C][D][D]          */
/* count--               →  [A][C][D]    (D2 е сляп)*/
```

КОД	ОБЯСНЕНИЕ
`for (int i = idx; i < *count - 1; i++)`	От позицията на изтрития до предпоследния.
`arr[i] = arr[i + 1];`	Преместваме десния наляво. Целият struct се копира.
`(*count)--;`	Намаляваме брояча. Последният дубликат вече не е видим.

Грешки характерни за Изпит 1

```
[NE]  int code;                      // препълване за 13-цифрен код!
[OK]  long long code;                // задължително

[NE]  scanf("%d", &tmp.code);       // %d е за int, не за long long
[OK]  fscanf(f, "%lld", &tmp.code); // %lld за long long

[NE]  arr[idx] = arr[idx+1]; count--; // само 1 преместване — грешно!
[OK]  for (i = idx; i < *count-1; i++) arr[i] = arr[i+1]; (*count)--;
```